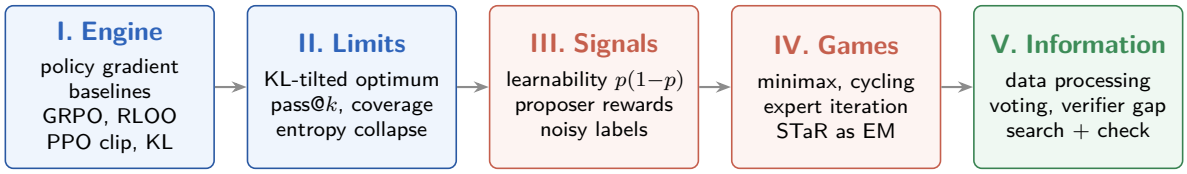
NOTES ▪ VERSION 1.3 ▪ NOVEMBER 2026

Foundations of Self-Play for LLM Reasoning

The theory needed to study how a model can learn to reason when no ground-truth answer is available

Chandak Chakma ▪ Personal study notes ▪ chandakchakma.github.io

Purpose. Challenger–Solver self-play asks a single model to pose its own problems, solve them, and judge its own answers. These notes collect the concepts, results and formulas that underlie this idea. Each topic is presented with a definition, a short derivation, a worked example with concrete numbers, and an account of its relevance to self-play. **Part V is the core of these notes:** it asks where new knowledge can come from in a loop that has no external teacher.



VI. Evidence: confidence intervals ▪ seeds ▪ unbiased pass@k ▪ same-compute baselines ▪ contamination **VII. Open problems:** where a new thread can lead

Reading guide. Parts I–II describe the optimization engine that every method shares. Part III treats reward design for the Challenger, and Part IV analyzes the dynamics of two interacting learners. Part V develops the theory of self-improvement, and Part VI sets out the standard of evidence that reviewers expect. Each part closes with exercises, whose solutions appear in Appendix B; Appendix A is a one-page formula sheet.

Notation

x	problem (prompt)	$\pi_\theta(y \mid x)$	Solver policy
y	full response, ending with a final answer	π_{ref}	frozen reference policy (base model)
z	reasoning trace (rationale)	$R(x, y)$	reward, typically in $\{0, 1\}$
$p(x)$	Solver's success probability on x	G	samples per problem (group size)
$\hat{A}_i$	advantage of sample i	$\beta,\ \varepsilon$	KL coefficient, clipping range
$\hat{p}$	agreement rate with the majority-vote answer	k	sample budget in pass@k

PART I

The RL engine

1 LLM generation as reinforcement learning

We treat a problem x as the state and the complete response y as a single action. A verifier (or, in self-play, a pseudo-label) assigns one scalar reward at the end of the response. This *bandit view* is the formulation adopted by GRPO, R-Zero and SOAR. The policy factorizes over tokens:

$$\pi_\theta(y \mid x) = \prod_{t=1}^{|y|} \pi_\theta(y_t \mid x, y_{<t}), \qquad J(\theta) = \mathbb{E}_{x\sim\mathcal{D}}\, \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}\big[R(x,y)\big].$$

With a binary reward, $J(\theta) = \mathbb{E}_x[p_\theta(x)]$, so training maximizes the success probability averaged over problems.

2 The policy gradient

Key result | the log-derivative trick

$$\nabla_\theta J(\theta) = \mathbb{E}_{x,\, y\sim\pi_\theta}\Big[R(x,y)\, \nabla_\theta \log \pi_\theta(y \mid x)\Big], \qquad \nabla_\theta \log \pi_\theta(y \mid x) = \sum_t \nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t}).$$

Derivation. For a fixed x,

$$\nabla_\theta \sum_y \pi_\theta(y \mid x) R(x,y) = \sum_y \pi_\theta(y \mid x)\, R(x,y)\, \frac{\nabla_\theta \pi_\theta(y \mid x)}{\pi_\theta(y \mid x)} = \mathbb{E}_{y\sim\pi_\theta}[R\, \nabla_\theta \log \pi_\theta(y \mid x)].$$

Because the result is an expectation under the policy, it can be estimated from samples, and the intractable sum over all possible responses never has to be computed.

Why it matters for self-play. Every method discussed in these notes (PPO, GRPO, RLOO, R-Zero, Absolute Zero, SOAR) estimates this same expectation. The methods differ only in how they weight each sample and in the source of the reward R.

3 Baselines and advantages

Key result | baselines introduce no bias

For any baseline $b(x)$ that does not depend on the sampled y,

$$\mathbb{E}_{y\sim\pi_\theta}\big[b(x)\, \nabla_\theta \log \pi_\theta(y \mid x)\big] = b(x) \sum_y \nabla_\theta \pi_\theta(y \mid x) = b(x)\, \nabla_\theta \sum_y \pi_\theta(y \mid x) = b(x)\, \nabla_\theta 1 = 0,$$

and therefore $\nabla_\theta J = \mathbb{E}\big[(R - b(x))\, \nabla_\theta \log \pi_\theta\big]$. The quantity $\hat{A} = R - b$ is called the *advantage*. A well-chosen baseline, close to $\mathbb{E}[R \mid x]$, substantially reduces the variance of the gradient estimate.

Worked example | why the baseline matters

Suppose four samples receive rewards $(1, 1, 1, 0)$. Without a baseline, the three correct samples are reinforced with weight 1, while the incorrect one receives weight 0 and is therefore never penalized.

With $b = 0.75$, the weights become $(+0.25, +0.25, +0.25, -0.75)$. The single failure now carries the largest signal, which is the desired behavior on an easy problem.

Pitfall. A baseline that incorporates the sample's *own* reward, such as a group mean that includes R_i, is no longer independent of y_i. The leave-one-out baseline introduced below resolves this issue.

4 Group-relative advantages: GRPO and RLOO

Definition | GRPO (Shao et al., 2024)

For each problem, G responses are sampled. The advantage of response i, which is shared by all of its tokens, is

$$\hat{A}_i = \frac{R_i - \mathrm{mean}(R_1, \ldots, R_G)}{\mathrm{std}(R_1, \ldots, R_G) + \epsilon_{\mathrm{std}}}.$$

The objective combines a PPO-style clipped ratio (Section 5) with a KL penalty toward π_{ref}:

$$\mathcal{J}_{\mathrm{GRPO}} = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|y_i|}\sum_{t}\min\Big(\rho_{i,t}\hat{A}_i,\ \mathrm{clip}(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon)\hat{A}_i\Big)\right] - \beta\, D_{\mathrm{KL}}(\pi_\theta \,\|\, \pi_{\mathrm{ref}}).$$

Definition | RLOO (Ahmadian et al., 2024)

$$\hat{A}_i = R_i - \frac{1}{G-1}\sum_{j \neq i} R_j.$$

The baseline is computed from the *other* samples only; it is therefore independent of y_i, and the resulting estimator is unbiased.

Key result | the two are nearly equivalent

$R_i - \frac{1}{G-1}\sum_{j\neq i} R_j = \frac{GR_i - \sum_j R_j}{G-1} = \frac{G}{G-1}\Big(R_i - \mathrm{mean}_j\, R_j\Big)$. Mean-centering therefore equals the RLOO advantage scaled by the constant $(G-1)/G$: the gradient direction remains unbiased, and only its scale changes. The substantive difference in GRPO is the division by the standard deviation.

Worked example | one success in four

Consider rewards $(1, 0, 0, 0)$, with mean 0.25 and standard deviation $\sqrt{0.25 \cdot 0.75} = 0.433$. GRPO gives $\hat{A} = (+1.73, -0.58, -0.58, -0.58)$. RLOO gives $\hat{A} = (1-0,\ 0-\frac{1}{3},\ 0-\frac{1}{3},\ 0-\frac{1}{3}) = (+1, -0.33, -0.33, -0.33)$, which is exactly $\frac{4}{3} \times (0.75, -0.25, -0.25, -0.25)$.

Pitfall. This formula contains three subtle pitfalls.

- If all G rewards in a group are equal, every advantage is 0 and the problem provides no learning signal. The *dynamic sampling* scheme of DAPO discards such groups (Yu et al., 2025).
- With binary rewards, $\mathrm{std} = \sqrt{p(1-p)}$; dividing by it therefore scales a problem's weight by $1/\sqrt{p(1-p)}$ relative to plain mean-centering. This up-weights problems that are solved either almost always or almost never.
- The factor $1/|y_i|$ reduces the per-token penalty on long incorrect responses, so the policy drifts toward longer incorrect answers. Dr. GRPO removes both normalizations (Liu et al., 2025).

5 Trust regions: PPO clipping and the KL penalty

Definition | the clipped surrogate (Schulman et al., 2017)

Given the per-token probability ratio $\rho_t = \pi_\theta(y_t \mid x, y_{<t})/\pi_{\text{old}}(y_t \mid x, y_{<t})$, the clipped surrogate objective is

$$L^{\text{CLIP}} = \mathbb{E}\Big[\min\left(\rho_t \hat{A},\ \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)\,\hat{A}\right)\Big].$$

Worked example | the effect of clipping

Let $\varepsilon = 0.2$. If $\hat{A} = +1$ and a token's probability has already increased by 30% ($\rho = 1.3$), then $\min(1.3, 1.2) = 1.2$ is a constant; the gradient vanishes, and the update no longer raises that token's probability. Similarly, if $\hat{A} = -1$ and $\rho = 0.7$, then $\min(-0.7, -0.8) = -0.8$, which is again constant. Clipping thus confines each update to a small trust region around π_{old}.

The KL term. GRPO estimates $D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$ at each token with the estimator $k_3 = r - 1 - \log r$, where $r = \pi_{\text{ref}}/\pi_\theta$ and the token is sampled from π_θ. Since $\mathbb{E}_{\pi_\theta}[r] = 1$, it follows that $\mathbb{E}[k_3] = \mathbb{E}[-\log r] = D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$; the estimator is therefore unbiased, and each individual estimate is non-negative.

Why it matters for self-play. R-Zero retains a KL anchor ($\beta = 0.01$), whereas Absolute Zero removes it. The anchor slows drift and collapse, but it also ties the policy to the base model, which is precisely what self-play aims to surpass (Section 6). Whether to anchor is therefore a substantive design decision rather than an implementation detail.

Exercises for Part I

Exercise 1. Compute the GRPO advantages for rewards $(1, 1, 1, 0)$. Which sample receives the largest weight, and why is this desirable?

Exercise 2. Show that GRPO without the standard-deviation normalization equals RLOO up to a constant factor. What is this factor for $G = 8$?

PART II

What RL can and cannot do

6 The KL-regularized optimum: why RL primarily sharpens

Key result | the tilted distribution

$$\max_{\pi} \; \mathbb{E}_{y\sim\pi}[R(x,y)] - \beta\, D_{\mathrm{KL}}(\pi \,\|\, \pi_{\mathrm{ref}}) \quad \Longrightarrow \quad \pi^\star(y \mid x) = \frac{\pi_{\mathrm{ref}}(y \mid x)\, e^{R(x,y)/\beta}}{Z(x)},$$

where $Z(x) = \sum_y \pi_{\mathrm{ref}}(y \mid x)\, e^{R(x,y)/\beta}$ makes the probabilities sum to one.

Derivation (Gibbs variational identity). The divergence from the optimum expands as $D_{\mathrm{KL}}(\pi\|\pi^\star) = \sum_y \pi \log \frac{\pi}{\pi^\star} = D_{\mathrm{KL}}(\pi\|\pi_{\mathrm{ref}}) - \frac{1}{\beta}\mathbb{E}_\pi[R] + \log Z$. Multiplying by $-\beta$ yields

$$\mathbb{E}_\pi[R] - \beta D_{\mathrm{KL}}(\pi\|\pi_{\mathrm{ref}}) = \beta \log Z - \beta\, D_{\mathrm{KL}}(\pi\|\pi^\star).$$

The left-hand side is the objective, and the right-hand side is maximized when $D_{\mathrm{KL}}(\pi\|\pi^\star) = 0$, that is, when $\pi = \pi^\star$. The same identity provides the closed form on which DPO is built (Rafailov et al., 2023).

Two consequences. (i) *Support is preserved:* if $\pi_{\mathrm{ref}}(y \mid x) = 0$, then $\pi^\star(y \mid x) = 0$. RL therefore re-weights only those responses that the base model can already produce. (ii) *Binary rewards multiply the odds:* given success probability p under π_{ref}, the optimal policy satisfies

$$\frac{p^\star}{1-p^\star} = \frac{p}{1-p}\, e^{1/\beta}.$$

Worked example | amplification and its limits

With $\beta = 0.1$, $e^{1/\beta} = e^{10} \approx 22{,}026$.

base success probability p	0.1	10^{-3}	10^{-6}
optimal $p^\star$	0.9996	0.957	0.022

At the optimum, a 1-in-1000 success rate under the base model rises to 96%. The optimum, however, is not what a training run reaches: with G samples per problem, the probability of observing even a single success is $1-(1-p)^G$, which for $G = 8$ and $p = 10^{-3}$ is only 0.008. In practice, RL therefore mainly amplifies behavior that the base model already produces frequently.

Why it matters for self-play. This formula condenses the "sharpen versus expand" debate into a single expression. Self-play operates through a different lever: the Challenger changes the *problems* x, constructing stepping stones on which $p(x)$ is not vanishingly small. Re-weighting the responses y alone cannot achieve this.

7 pass@k, coverage and the sharpening signature

Definition | pass@k and its unbiased estimator (Chen et al., 2021)

For a single problem, $\text{pass@}k = 1 - (1-p)^k$. Given $n \geq k$ samples of which c are correct, the unbiased estimator is

$$\widehat{\text{pass@}k} = 1 - \binom{n-c}{k} \Big/ \binom{n}{k}.$$

Justification. If a random subset of k of the n samples is drawn, the subset fails only if all k of its members are incorrect, which occurs with probability $\binom{n-c}{k}/\binom{n}{k}$.

Worked example | computing the estimator

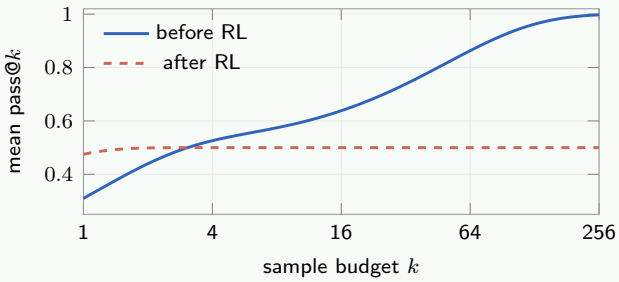
Let $n = 10$, $c = 2$ and $k = 5$. Then $\binom{8}{5}/\binom{10}{5} = 56/252 = 0.222$, so pass@5 $= 0.778$. The plug-in estimate $1 - (1-0.2)^5 = 0.672$ is biased downward in expectation, because $(1-\hat{p})^k$ is convex in $\hat{p}$ (by Jensen's inequality).

Worked example | the sharpening signature

Consider two problems. Training raises problem A from $p = 0.60$ to 0.95 and lowers problem B from $p = 0.02$ to 0.

	before	after
pass@1	0.31	0.475
pass@256	0.997	0.500

averaged over problems A and B



For problem B before training, $1 - 0.98^{256} = 1 - e^{256 \ln 0.98} = 1 - e^{-5.17} = 0.994$. Thus pass@1 rises while pass@256 falls: the model has become more reliable on problems it already solved, and has lost the ability to solve problems it previously solved only occasionally. The two curves cross between $k = 3$ and $k = 4$. Yue et al. (2025) report this pattern for RLVR at large k.

Why it matters for self-play. Evaluations should report pass@k up to large k (256 or more) and should count *newly solved* problems, that is, problems with base pass@256 $= 0$ that the trained model solves. Only such problems justify the word "expand".

8 Entropy and exploration

The policy entropy $H(\pi(\cdot \mid x)) = -\sum_y \pi(y \mid x) \log \pi(y \mid x)$ measures the diversity of the sampled responses. As RL sharpens the policy, entropy decreases: the samples within each group become homogeneous, the advantages become 0, and learning stops. This failure mode is known as *entropy collapse*. Common remedies include a larger upper clipping range ("clip-higher" in DAPO, with $\varepsilon_{\text{high}} > \varepsilon_{\text{low}}$), an entropy bonus, a higher sampling temperature, and discarding zero-variance groups. Section 11 shows that a majority-vote reward is itself an entropy-minimizing objective.

Exercises for Part II

Exercise 3. With $\beta = 0.2$ and base success probability $p = 0.01$, what is $p^\star$ at the KL-regularized optimum?

Exercise 4. Evaluate the unbiased estimator for $n = 16$, $c = 1$ and $k = 8$. Then show that with a single correct sample ($c = 1$) the estimate always equals k/n.

PART III

Learning signals: what should the Challenger propose?

9 Learnability

Key result | the $p(1-p)$ law

For a binary reward with success probability p, the variance is $\mathrm{Var}[R] = p(1-p)$, which attains its maximum $(1/4)$ at $p = 1/2$. A GRPO group of size G provides a learning signal only if its rewards are not all equal, so the probability of an informative group is

$$\Pr[\text{informative group}] = 1 - p^G - (1-p)^G.$$

Worked example | $G = 8$

p (or $1-p$)	0.5	0.9	0.99
fraction of informative groups	0.992	0.570	0.077

A problem that the Solver solves 99% of the time yields a learning signal in only about 1 group in 13. This arithmetic underlies the familiar prescription to "generate problems at the edge of the Solver's ability". R-Zero motivates the same target through a lower bound on the Solver's remaining learning potential, which increases with $\hat{p}(1-\hat{p})$.

Pitfall. The law concerns the *true* success rate. In label-free self-play, we observe only the agreement rate $\hat{p}$ with a pseudo-label, which coincides with p only when the pseudo-label is correct.

10 Proposer rewards: a comparison

Method	Challenger (proposer) reward	Ground truth from	Main limitation
Asymmetric self-play (2017)	$r_A = \gamma \max(0, t_B - t_A)$, $r_B = -\gamma t_B$ (completion times)	environment	requires a reversible environment
PAIRED (2020)	regret $= U(\pi_{\text{antagonist}}) - U(\pi_{\text{protagonist}})$	environment reward	requires a strong antagonist
Absolute Zero (2025)	$1 - \bar{r}$ if $0 < \bar{r} < 1$, else 0 ($\bar{r}$ = solve rate)	Python executor	restricted to code
R-Zero (2025)	$\max\big(0,\ 1 - 2\lvert\hat{p} - \tfrac{1}{2}\rvert - \lambda\lvert C_k\rvert/B\big)$	none (majority vote)	pseudo-label accuracy decays
SOAR (2026)	$\mathrm{Perf}_H(\mathrm{Train}(\theta, D_\phi)) - \mathrm{Perf}_H(\theta)$	labeled hard set H	requires labels on H
Learning progress (2007)	reduction of the learner's error over time	environment	requires a measurable error

SOAR (Sundaram, Kempe and Ollivier, ICML 2026) is the current peer-reviewed frontier. Its formulation is bilevel: the teacher generates a problem set D_ϕ, the student trains on it briefly, and the teacher is rewarded by the student's measured improvement on hard problems that the student initially could not solve (0 of 128 samples correct). The paper finds that grounded rewards outperform learnability rewards, and that the well-posedness of generated questions matters more than the correctness of their

answers. Its principal limitation is the labeled hard set H. **PowerPlay** (Schmidhuber, 2011) is the conceptual precursor: it continually searches for the simplest problem that the current solver cannot yet solve.

Worked example | a blind spot in R-Zero's reward

Suppose the Solver produces ten answers: four give 20, four give 22, and two contain no boxed answer. Then $\hat{p} = 4/10$ and $r_{\text{unc}} = 1 - 2|0.4 - 0.5| = 0.8$, a high reward, and the question passes the 3–7 filter. In the released code, the pseudo-label is whichever of 20 and 22 appeared first (Python's `max` returns the first maximal key), and each unboxed answer is counted as a vote for the string `"None"`; the question is discarded later only if `"None"` wins the vote. The reward therefore cannot distinguish a two-way split from scattered guessing, and the choice between the two candidates is effectively arbitrary. Reported behavior should be checked against the released code.

Why it matters for self-play. Rewards constructed solely from the Solver's agreement measure difficulty rather than correctness. Rewards grounded in genuine progress (SOAR, PAIRED) require a source of ground truth. Obtaining a grounded signal *without* labels remains the central open problem.

11 Learning from noisy labels

Key result | random noise only attenuates the signal

Suppose the observed reward $\tilde{R}$ flips the true reward with probability η, independently of the sampled response. Then $\mathbb{E}[\tilde{R} \mid y] = \eta + (1 - 2\eta)R(y)$, and since $\mathbb{E}[\nabla_\theta \log \pi_\theta] = 0$,

$$\mathbb{E}\big[\tilde{R}\, \nabla_\theta \log \pi_\theta\big] = (1 - 2\eta)\, \nabla_\theta J.$$

Whenever $\eta < 1/2$, the expected gradient retains its direction; only its magnitude is reduced.

Key result | correlated noise biases the signal

If the label error depends on the response, we write $\tilde{R} = R + \delta(x, y)$. Then $\mathbb{E}[\tilde{R}\, \nabla_\theta \log \pi_\theta] = \nabla_\theta J + \mathbb{E}[\delta\, \nabla_\theta \log \pi_\theta]$, where the second term is a systematic drift toward whichever responses δ favors.

Worked example | the majority-vote reward

The pseudo-label is the mode $m(x)$ of the Solver's own samples, and the reward is $\mathbf{1}[\text{answer}(y) = m(x)]$. Its expected value is $\pi_\theta(m \mid x)$, which is maximized by placing all probability mass on m, regardless of whether m is correct. For instance, let $\pi(20) = 0.40$, $\pi(22) = 0.35$, with the remaining answers sharing 0.25, and let the correct answer be 22. The vote selects 20 as the pseudo-label, and training drives $\pi(20)$ toward 1: the model converges to a confident but incorrect answer.

Why it matters for self-play. This mechanism explains the characteristic "rise, then fall" pattern. Early in training the mode is usually correct (R-Zero's pseudo-labels are 79% accurate in round 1), so sharpening is beneficial. As the Challenger generates harder questions, the mode is incorrect more often (63% accurate by round 3), and the same sharpening begins to reinforce errors.

Exercises for Part III

Exercise 5. For $G = 4$ and $p = 0.75$, what fraction of groups is informative?

Exercise 6. Under symmetric label noise with $\eta = 0.3$, by what factor is the expected gradient scaled? What happens at $\eta = 0.5$?

PART IV
Self-play as a game

12 Games, equilibria, and cycling in self-play

Definition | zero-sum games and equilibria

In a zero-sum game, player 1 receives payoff $u(a, b)$ and player 2 receives $-u(a, b)$. For mixed strategies s, t and payoff matrix U, von Neumann's minimax theorem states that $\max_s \min_t s^\top U t = \min_t \max_s s^\top U t$. A *Nash equilibrium* is a pair of strategies from which neither player can improve its payoff by deviating unilaterally.

Key result | naive self-play can diverge

Consider $f(u, v) = uv$, where u is the minimizing player and v the maximizing player. Simultaneous gradient steps $u_{t+1} = u_t - \eta v_t$, $v_{t+1} = v_t + \eta u_t$ yield

$$u_{t+1}^2 + v_{t+1}^2 = (1 + \eta^2)\,(u_t^2 + v_t^2),$$

so the iterates spiral outward from the equilibrium $(0, 0)$ for every nonzero step size.

In rock–paper–scissors, best responses cycle indefinitely: rock is answered by paper, paper by scissors, and scissors by rock. Standard remedies include the following.

- **Fictitious play:** each player best-responds to the *average* of all past opponent strategies; in two-player zero-sum games, the time-averaged strategies converge (Robinson, 1951).
- **Opponent pools:** the agent plays against a randomly selected earlier checkpoint, as in AlphaGo.
- **PSRO** (Lanctot et al., 2017): a growing population of policies is maintained and mixed according to a Nash equilibrium of the meta-game.
- **Optimistic or extragradient updates:** these methods converge on bilinear games, on which plain gradient steps spiral outward.

Why it matters for self-play. Challenger–Solver self-play is general-sum and non-stationary: each player's objective shifts as the other learns. The forgetting of earlier skills, which R-Diverse reports as cross-round mode cycling, is the LLM counterpart of this cycling. Memory banks and policy populations are the LLM analogue of fictitious play.

13 Search as a teacher: expert iteration and AlphaZero

Expert iteration (Anthony et al., 2017) alternates between two steps. First, an *improvement operator* $\mathcal{I}$, such as tree search, filtering, or voting, transforms the current policy into a stronger one; second, the policy is trained to imitate the improved policy, $\pi \leftarrow \arg\min_\pi D_{\mathrm{KL}}(\mathcal{I}(\pi) \,\|\, \pi)$. AlphaZero is the best-known instance, with the loss

$$\ell = (z - v)^2 - \boldsymbol{\pi}_{\mathrm{MCTS}}^\top \log \mathbf{p} + c\,\|\theta\|^2,$$

where z is the actual game outcome, v the predicted value, $\boldsymbol{\pi}_{\mathrm{MCTS}}$ the search's move distribution, and $\mathbf{p}$ the network's.

Key result | the condition for self-improvement

The loop yields improvement only if $\mathcal{I}(\pi)$ is reliably better than π. In Go, search combined with the true game outcome ensures this. In label-free LLM self-play, the operator is typically a majority vote, which improves on the policy only when the mode is correct.

14 STaR as maximum marginal likelihood

Key result | Fisher's identity

For a latent rationale z and gold answer $y^\star$,

$$\nabla_\theta \log p_\theta(y^\star \mid x) = \mathbb{E}_{z\sim p_\theta(z|x,y^\star)}\Big[\nabla_\theta \log p_\theta(z, y^\star \mid x)\Big].$$

Derivation. $\nabla_\theta \log p(y^\star \mid x) = \frac{1}{p(y^\star|x)} \sum_z \nabla_\theta p(z, y^\star \mid x) = \sum_z \frac{p(z,y^\star|x)}{p(y^\star|x)} \nabla_\theta \log p(z, y^\star \mid x)$, where the weight is precisely the posterior $p(z \mid x, y^\star)$. If the answer is a deterministic function of the rationale, $y = \mathrm{ans}(z)$, then sampling $z \sim p_\theta(z \mid x)$ and retaining it only when $\mathrm{ans}(z) = y^\star$ yields exact samples from the posterior. Fine-tuning on the retained rationales, as in STaR (Zelikman et al., 2022), therefore follows a stochastic gradient of $\log p_\theta(y^\star \mid x)$; the procedure is an EM-style algorithm.

Key result | RL versus likelihood

RL with a $0/1$ reward maximizes $p_\theta(y^\star \mid x)$, with $\nabla_\theta p = p \nabla_\theta \log p$, whereas STaR maximizes $\log p_\theta(y^\star \mid x)$, with $\nabla_\theta \log p = \nabla_\theta p / p$. On a problem solved 1% of the time, the likelihood objective therefore weights the gradient 100 times more heavily than RL does; log-likelihood thus emphasizes hard problems.

Why it matters for self-play. The choice between maximizing p and maximizing $\log p$ determines whether rare successes on hard problems drive learning. It is therefore directly connected to the Challenger's role of keeping p within the learnable range.

Exercises for Part IV

Exercise 7. In the bilinear example with $\eta = 0.1$ and initial point $(u, v) = (1, 0)$, what is the distance of the iterates from the equilibrium after 10 steps?

Exercise 8. Consider a problem with $p_\theta(y^\star \mid x) = 0.05$. By what factor does the likelihood gradient exceed the RL gradient on this problem?

PART V

Where can new knowledge come from?

15 The data processing inequality

Key result | processing cannot create information

If $T \to X \to S$ forms a Markov chain, that is, if the signal S is computed from X alone, then

$$I(T;S) \le I(T;X).$$

Let T denote the truth, X the problem together with the model, and S any label-free signal computed from the model's own samples, such as votes, confidence scores, or consistency checks. The inequality states that no such signal carries more information about the truth than the model already holds. Label-free self-play can therefore *extract and concentrate* this information, but it cannot *create* it. For the loop to expand what the model knows, it requires an external source of information (an executor, a proof checker, or an environment) or an asymmetry, for instance a task on which checking is easier than solving.

An important qualification. The inequality bounds information, not usability. A model may "know" an answer in the sense of pass@256 and still fail at pass@1. Converting such latent knowledge into reliable behavior has genuine value, but it constitutes sharpening rather than the acquisition of new knowledge.

16 The generator–verifier gap

Key result | filtering helps only if the check is informative

Suppose self-generated answers are filtered by the model's own check, with true positive rate $\mathrm{TPR} = \Pr(\text{accept} \mid \text{correct})$ and false positive rate $\mathrm{FPR} = \Pr(\text{accept} \mid \text{wrong})$. By Bayes' rule, the precision, that is, the accuracy among accepted answers, is

$$\mathrm{prec} = \frac{p\,\mathrm{TPR}}{p\,\mathrm{TPR} + (1-p)\,\mathrm{FPR}}, \qquad \text{and} \qquad \mathrm{prec} > p \iff \mathrm{TPR} > \mathrm{FPR}.$$

Proof. A direct rearrangement gives $\mathrm{prec} > p \iff \mathrm{TPR} > p\,\mathrm{TPR} + (1-p)\mathrm{FPR} \iff (1-p)\mathrm{TPR} > (1-p)\mathrm{FPR}$.

Worked example | an informative and an uninformative verifier

With $p = 0.3$, $\mathrm{TPR} = 0.9$, and $\mathrm{FPR} = 0.2$, we obtain $\mathrm{prec} = 0.27/(0.27 + 0.14) = 0.659$, more than double the base accuracy, so training on the accepted answers can help. If instead $\mathrm{TPR} = \mathrm{FPR}$, then $\mathrm{prec} = p$ and filtering provides no benefit.

Song et al. (2025) formalize this generation–verification gap and study how it varies across models and verification methods. Their analysis offers the cleanest characterization of *when* self-improvement is possible at all.

Why it matters for self-play. A majority vote is itself a verifier: its TPR and FPR are determined by how often correct and incorrect answers become the mode. When errors are shared, a wrong answer becomes the mode as readily as a correct one, and the gap closes.

17 When does voting recover the truth? A theory of shared errors

Majority voting serves as the default answer key in label-free self-play. This section asks precisely when the vote is correct, why it fails, and what any replacement must do better. Together with Section 18, it is the most important part of these notes for the design of a new method: it concludes with a map of the channels that a new signal can exploit (Section 17.8) and a worksheet for evaluating a candidate signal (Section 17.9).

17.1 Independent voters: Condorcet's theorem and its rate

Key result | Condorcet's jury theorem

Consider n independent voters on a two-option question, each correct with probability q. Then

$$P_{\text{maj}}(n) = \sum_{k>n/2} \binom{n}{k} q^k (1-q)^{n-k} \xrightarrow[n\to\infty]{} \begin{cases} 1, & q > 1/2 \\ 0, & q < 1/2. \end{cases}$$

Rate of convergence. For $q > 1/2$, Hoeffding's inequality gives $\Pr[\text{majority wrong}] \le \exp(-2n(q-\frac{1}{2})^2)$, so the error decays exponentially in the number of voters.

Worked example | independent voters

For $q = 0.6$ and $n = 9$, the terms for $k = 5, \dots, 9$ are $0.251,\ 0.251,\ 0.161,\ 0.060,\ 0.010$, which gives $P_{\text{maj}} = 0.733$. Nine weak voters thus outperform a single one. For $n = 101$, the bound limits the error to at most 0.133, whereas the exact error is 0.021.

17.2 Many answers: the plurality vote converges to the mode

Key result | plurality voting

Let $\pi(a \mid x)$ denote the Solver's answer distribution and t the true answer. As $n \to \infty$, the plurality vote returns the mode $\arg\max_a \pi(a \mid x)$, provided that the mode is unique. For each wrong answer w with $\pi(w \mid x) < \pi(t \mid x)$,

$$\Pr\left[\, w \text{ gets at least as many votes as } t \,\right] \le \exp\Big(-\tfrac{n}{2} \left(\pi(t \mid x) - \pi(w \mid x)\right)^2 \Big),$$

and a union bound over the wrong answers then bounds the probability that the vote fails to return t.

Derivation. Assign each sample a score of $+1$ if it answers t, -1 if it answers w, and 0 otherwise. The mean score is $\pi(t \mid x) - \pi(w \mid x)$, and each score lies in an interval of length 2, so Hoeffding's inequality bounds the probability that the sum is ≤ 0 by $\exp(-2n\Delta^2/2^2) = \exp(-n\Delta^2/2)$.

Two consequences. (i) In the limit, the vote is correct exactly when the truth is the mode; additional samples only yield a better estimate of the mode. (ii) The *margin* $\Delta = \pi(t \mid x) - \max_{w \neq t} \pi(w \mid x)$ determines how many samples are required.

Worked example | a small margin leaves the vote close to chance

Suppose the true answer 22 has $\pi = 0.40$, the wrong answer 20 has 0.35, and five other answers have 0.05 each. With 10 votes, the plurality vote selects 22 only 56% of the time and 20 in 42% of cases (computed by exact multinomial enumeration, with ties split evenly). For this pair, the bound guarantees 90% accuracy only after about $1{,}800$ votes. R-Zero's reward is blind to this risk: it depends only on the share of the top answer ($\hat{p} \approx 0.4$, reward ≈ 0.8), not on how close the

runner-up is.

17.3 Three models of correlated errors

Across questions, votes are not independent. We consider three models of how they depend on one another; for a single LLM, the third model is exact.

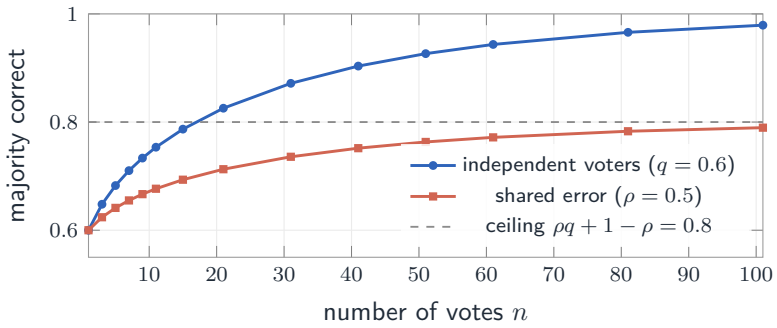
Key result | (a) common shock: a shared error caps the vote

With probability ρ, all voters copy a single shared draw, which is correct with probability q; otherwise, they vote independently. Then

$$P = \rho\, q + (1-\rho)\, P_{\text{maj}}(n) \xrightarrow[n\to\infty]{} \rho\, q + (1-\rho) < 1.$$

Worked example | correlated voters

For $\rho = 0.5$ and $q = 0.6$, we have $P = 0.3 + 0.5 \times 0.733 = 0.667$ at $n = 9$, and the accuracy never exceeds 0.80, however many samples are drawn. Additional samples cannot remove a shared error.



Accuracy under independent votes approaches 1, whereas votes that share an error remain below the ceiling.

Key result | (b) exchangeable voters: the effective number of votes

If every pair of correctness indicators has correlation $r > 0$, then $\text{Var}(\text{mean}) = \frac{q(1-q)}{n}(1+(n-1)r)$, so the vote behaves like

$$n_{\text{eff}} = \frac{n}{1+(n-1)\,r} \xrightarrow[n\to\infty]{} \frac{1}{r}$$

independent votes, as measured by the variance of the vote share. This quantity is the *design effect* of survey sampling (Kish 1965); it serves as a rough guide rather than an exact formula for accuracy. With $r = 0.2$, ten samples are worth about 3.6 independent ones, and a thousand are worth at most five.

Key result | (c) latent difficulty: the ceiling of voting

Samples are independent *given the question*, but each question x has its own success rate $q(x)$. The accuracy of the vote as $n \to \infty$ is then

$$\Pr_x\left[q(x) > \tfrac{1}{2}\right] \text{ (two options)}, \qquad \text{or} \qquad \Pr_x\left[\text{the mode of } \pi(\cdot \mid x) \text{ is the truth}\right] \text{ (in general)}.$$

The fraction of questions whose mode is correct is therefore a hard ceiling for any vote.

Worked example | voting lifts easy questions and sinks hard ones

Suppose 60% of questions have $q = 0.8$, 25% have $q = 0.45$, and 15% have $q = 0.1$. A single sample is correct $0.6 \cdot 0.8 + 0.25 \cdot 0.45 + 0.15 \cdot 0.1 = 60.75\%$ of the time, while an infinite vote is correct 60% of the time. The vote raises the easy questions to 100% and drives the remaining 40% to 0%, even though the model sometimes solves them.

Key result | (c) explains (b)

For a single LLM sampled at a fixed prompt, model (c) is exact: samples are independent given the question, so all of their correlation arises through the question itself. Averaged over questions, model (c) yields exactly the correlation of model (b):

$$r = \frac{\mathrm{Var}_x[q(x)]}{\bar{q}\,(1-\bar{q})}, \qquad \bar{q} = \mathbb{E}_x[q(x)].$$

In the preceding example, $r = 0.067/0.238 = 0.28$.

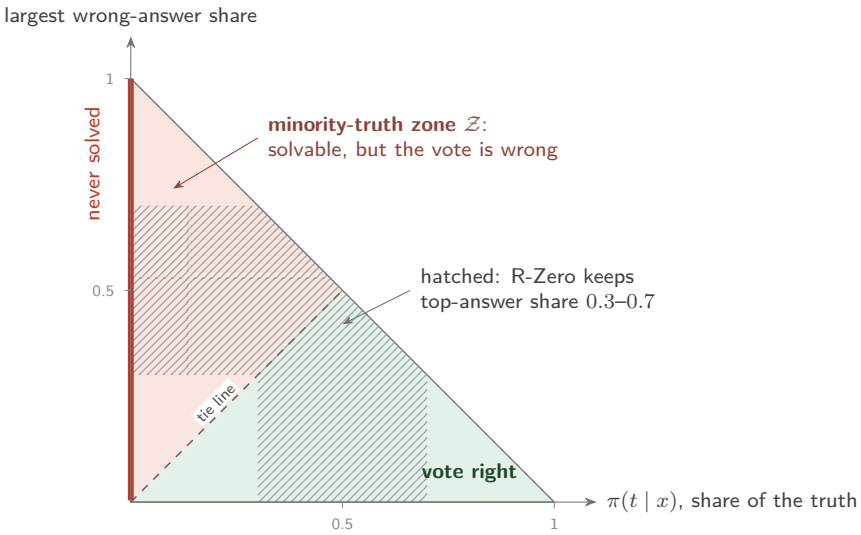
Derivation. For two samples $i \neq j$ drawn on the same random question, $\mathrm{Cov}(X_i, X_j) = \mathbb{E}[q^2] - \bar{q}^2 = \mathrm{Var}_x[q]$ and $\mathrm{Var}(X_i) = \bar{q}(1-\bar{q})$.

A "shared error" of a single model is therefore not noise between its samples; it is a *question whose mode is wrong*. Ensembles do not escape this problem either: Kim et al. (2025) find that larger and more accurate models make highly correlated errors, even across different providers.

Definition | the minority-truth zone

$$\mathcal{Z} = \Big\{ x : \; 0 < \pi(t \mid x) \leq \max_{w \neq t} \pi(w \mid x) \Big\}.$$

On $\mathcal{Z}$, the model can solve the problem, yet the vote is wrong (or, at a tie, correct only by chance). A better label pays off *only* here: outside $\mathcal{Z}$ the vote is already correct, and on questions that are never solved, a correct label gives an outcome-reward learner no correct sample to reinforce.



Each question corresponds to a point. R-Zero's filtering band straddles the tie line, so it can retain questions from $\mathcal{Z}$.

Pitfall. R-Zero retains questions whose top answer receives 30–70% of the votes, and its Challenger is rewarded most near 50%. This band crosses the tie line, so the retained set can contain many

minority-truth questions, and a stronger Challenger can find more of them. This is consistent with the decline in R-Zero's pseudo-label accuracy from 79% to 63% over three rounds.

17.4 The indistinguishability lemma

Key result | a rule cannot separate what it cannot observe

Let a labeling rule L have access only to an observation $O(x)$ (for example, the final answers of n samples, their full reasoning traces, or hidden states) together with randomness independent of the truth. If two problems x and x' induce the same distribution for $O(x)$ and $O(x')$ but have different truths ($t \neq t'$), then

$$\Pr[L(x) = t] + \Pr[L(x') = t'] \leq 1.$$

Majority voting is the special case in which $O(x)$ consists of the final answers of samples from $\pi(\cdot \mid x)$.

Proof. The labels $L(x)$ and $L(x')$ share the same distribution D over labels, and $D(t) + D(t') \leq 1$ because $t \neq t'$.

Corollary: three escape routes. A labeling rule that outperforms the vote must observe something whose distribution *differs* between such a pair. There are three ways to observe more:

1. **The model on other inputs** $\tilde{x}$, such as related problems, perturbations, inverse problems, or a rephrasing of x. This helps only if the model behaves differently on $\tilde{x}$ and $\tilde{x}'$.
2. **The model's internal states** $h(x)$, that is, hidden activations rather than outputs alone. This helps only if $h(x)$ differs between the two problems and encodes the truth.
3. **Information from outside the model**, such as execution, an environment, or a program that reads the problem text directly. This channel differs between the pair by design.

Worked example | why consistency checks cannot detect a consistent misreading

Suppose the model reads x as a different problem $\hat{x}$ and solves $\hat{x}$ well. Then on x, and on every related input (other parameter values, paraphrases that preserve the trigger, the inverse problem), it behaves exactly as it would on the correctly read $\hat{x}$-versions. Route 1 is therefore closed: each consistency check verifies $\hat{x}$ perfectly, so votes, regularities across problem families, and backward checks all agree with the wrong answer. Only a procedure that compares the model's written setup with the original text, or an external solver, can detect the discrepancy.

Why it matters for self-play. The lemma supplies the most informative first test for a new idea. Before running any experiment, we should state exactly what the signal observes, construct two problems on which that observation looks the same but whose truths differ, and ask whether the signal can tell them apart. If it cannot, no amount of tuning can compensate.

17.5 Sources of shared errors

Source	Shared through	Can be exposed by	Cannot be exposed by
Misreading the problem	the model's interpretation of the text	checking the written setup against the original text; an external solver; internal states, if they encode the reading	votes; checks that reuse the same reading (problem families, inverse problems)
Incorrect method or formula	the learned procedure	execution; a genuinely different method	votes; checks that use the same method
Missing knowledge	the pretraining data	external sources (tools, retrieval)	any signal internal to the model
Systematic arithmetic error	computational habits	executing the arithmetic	votes
Wording-induced bias	the surface form of the prompt	paraphrasing, reordering	–
Answer-extraction artifact	the answer parser	improved measurement; counting non-answers separately	–

17.6 Classical remedies and their limits for a single model

Dawid–Skene (1979). This model describes each annotator j by a confusion matrix and assumes that the annotators' errors are independent *given the truth*: $\Pr(\ell_1, \dots, \ell_m \mid t) = \prod_j \pi_j(\ell_j \mid t)$. EM then estimates both the truth and the reliability of every annotator. When annotators share an error, however, EM learns the shared error as the truth.

Surprisingly popular (Prelec, Seung and McCoy, 2017). Each voter provides an answer *and* a prediction of how others will answer; the method then selects the answer that is more popular than predicted. When asked "Is Philadelphia the capital of Pennsylvania?", most people answer yes, which is wrong. Those who know that the capital is Harrisburg also expect most people to answer yes, so "no" turns out to be more popular than predicted and is selected. The Bayesian truth serum (Prelec, 2004) is a closely related scoring rule that rewards answers for being surprisingly common.

Pitfall. Both remedies require voters whose errors are not identical. Recent work adapts them to LLMs: aggregation across *several different* LLMs using second-order information (2510.01499), and the truth serum as a GRPO reward, applied there to counter sycophancy (2608.25267). For a single model, however, all samples come from one mind, so its predictions about "other voters" may share its own error. Whether such predictions carry independent information remains an open question.

17.7 When does a new signal improve on the vote?

Key result | the override condition

Let e denote the error rate of the vote, and suppose a new signal S overrides the vote whenever the two disagree. Define the rescue rate $a = \Pr[S \text{ gives the truth} \mid \text{vote wrong}]$ and the break rate $b = \Pr[S \text{ overrides with a wrong answer} \mid \text{vote right}]$. Then

$$\Delta\text{acc} = e\,a - (1-e)\,b > 0 \iff \frac{a}{b} > \frac{1-e}{e}, \qquad \text{ER} = \frac{\Delta\text{acc}}{e} = a - b\,\frac{1-e}{e}.$$

Derivation. Overriding changes the outcome only where S and the vote disagree: it gains on the

incorrect votes it rescues ($e\,a$) and loses on the correct votes it breaks ($(1-e)\,b$).

Worked example | the worse the vote, the easier it is to improve

vote error e	0.21 (R-Zero, round 1)	0.37 (round 3)	0.50
required ratio a/b	3.76	1.70	1.00

A signal too weak to help in round 1 can become beneficial by round 3; new signals matter most where the vote is weakest.

Worked example | unsafe, and safe but weak

Suppose the vote is wrong on a fraction $e = 0.3$ of the questions, so that an override requires $a/b > 0.7/0.3 = 2.33$. A signal that rescues $a = 6\%$ of the incorrect votes but breaks $b = 3.5\%$ of the correct ones has $a/b = 1.71 < 2.33$, and accuracy declines: $\Delta\text{acc} = 0.3 \times 0.06 - 0.7 \times 0.035 = -0.0065$. A stricter variant that overrides only near-ties might rescue the same $a = 6\%$ while breaking only $b = 1.8\%$. Its ratio $a/b = 3.33$ now exceeds the threshold, yet it gains only 0.54 points ($\text{ER} = 0.018$). Both quantities therefore matter: the ratio determines safety, and the rescue rate inside $\mathcal{Z}$ determines the size of the gain.

17.8 Channels of new information: a map

Channel	Additional information	Defeats a consistent misreading?
More samples, same model	none	no
Other checkpoints or seeds	little (same lineage)	rarely
Training dynamics	how quickly an answer is learned	unclear
Perturbations, paraphrases, resampled numbers	the model on related inputs (route 1)	only if the error depends on surface form
Problem families and regularities	the model on related inputs (route 1)	no
Backward or inverse problems	the model on inverse inputs (route 1)	not if the reading is reused
Meta-predictions (surprisingly popular, truth serum)	beliefs about other voters	unclear for a single model
Internal states (probes, latent knowledge)	hidden activations (route 2)	possibly, since states can differ when outputs agree
Execution (code, proof checkers)	external to the model (route 3)	yes, for whatever can be executed
Other models (judges, ensembles)	another model	only if their errors differ

Route 2 is supported by genuine evidence: the internal representations of models often encode whether an answer is correct even when the output is wrong (Orgad et al., 2024), and contrast-consistent search identifies truth-like directions without labels (Burns et al., 2022). Two cautions apply. First, most probes are trained with labels (contrast-consistent search is the label-free exception). Second, a Challenger rewarded against a probe may learn to fool it. Any probe should also be compared against the model's own token probabilities, a free baseline that often carries the same signal.

17.9 A worksheet for a new signal

1. **Name the shared cause** that the signal is meant to overcome (for example, a consistent misreading).
2. **Construct the indistinguishable pair** on paper (Section 17.4): two problems on which everything the signal observes looks the same, but whose truths differ.
3. **Identify the channel** (route 1, 2 or 3) and verify that it differs on the pair. If it does not, stop.
4. **Check the ceiling**: on vote-wrong questions, count how often the truth appears among the samples at all. No answer key can correct more than that share; if it is low, the bottleneck is finding the answer, not judging it (Section 18).
5. **Measure a and b** on a small labeled pilot, separately on vote-wrong and vote-right questions. Require $a/b > (1-e)/e$ together with a meaningful error reduction, with both thresholds fixed before inspecting the results.
6. **Check for decay**: measure a by difficulty bin and by round; the rescue rate must hold up where the vote is weakest.
7. **Check the costs**: the signal must use no labels and no judge model, and it must be cheap enough to run on every question in every round.
8. **Audit novelty** before launching any training run.

Why it matters for self-play. The section reduces to six principles. (1) Additional samples never correct a wrong mode. (2) New information pays off only in the minority-truth zone. (3) A signal cannot separate two problems whose inputs to it look the same. (4) When the reading is wrong, consistency checks verify the wrong problem perfectly. (5) A useful signal needs both a high rescue-to-break ratio and a high rescue rate inside $\mathcal{Z}$. (6) No answer key can select an answer that was never sampled; when the truth is missing, the remedy is search, not a better judge.

18 Search and correctness: finding and recognizing rare answers

Section 17 showed that majority voting cannot correct a question whose mode is wrong. A stronger observation compounds the problem: when the vote is wrong, the correct answer is often absent from the samples altogether. For GPT-3.5 on math word problems, about 40% of failures contain no correct chain among the samples (FOBAR, Section 18.6); weaker models fare worse, and the correct answers that do appear are often sampled with probability 1% or lower (Section 18.3). The next question therefore has two parts, corresponding precisely to the two capabilities that AlphaGo's search possessed and that label-free self-play lacks: we must **find** the correct answer and then **recognize** it without an answer key.

Key result | one question, two steps

Let p denote the probability that a single sample is correct and k the number of samples, and suppose a check accepts a correct answer with probability TPR and an incorrect one with probability FPR. Then

$$\Pr[\text{found at least once}] = 1-(1-p)^k, \qquad k_{90\%} = \frac{\ln 10}{-\ln(1-p)} \approx \frac{2.3}{p}$$

where the second expression is the number of samples needed to find the correct answer with probability 0.9. An accepted sample is correct with probability

$$\mathrm{prec} = \frac{p\,\mathrm{TPR}}{p\,\mathrm{TPR} + (1-p)\,\mathrm{FPR}} \qquad \text{(Section 16).}$$

Derivation. All k samples miss with probability $(1-p)^k$; setting $(1-p)^k = 0.1$ and taking logarithms yields $k_{90\%}$, and the approximation follows from $-\ln(1-p) \approx p$ for small p. The precision follows from Bayes' rule, as in Section 16.

Worked example | why 10 samples do not suffice

For a question the model solves 5% of the time, 10 samples contain the correct answer with probability $1 - 0.95^{10} = 0.40$. Finding it with probability 90% requires $\ln 10/(-\ln 0.95) = 45$ samples, and at $p = 1\%$ the requirement rises to 230. A question with $p = 0$ is never found by sampling, for any k; only a different procedure (a hint, an easier stepping-stone problem, a decomposition, or new information) can reach it (Section 15).

18.1 The base-rate trap: rare answers require strict checks

Key result | how strict the check must be

Accepted answers are correct at least as often as not, $\mathrm{prec} \geq \frac{1}{2}$, if and only if

$$\mathrm{FPR} \leq \frac{p}{1-p}\,\mathrm{TPR} \;\approx\; p\,\mathrm{TPR}.$$

That is, the check's false-positive rate must be *smaller than the rate at which the correct answer appears*.

Derivation. Rearranging the definition of precision gives $\mathrm{prec} \geq \frac{1}{2} \iff p\,\mathrm{TPR} \geq (1-p)\,\mathrm{FPR}$.

Worked example | a seemingly strong check that fails

Suppose the correct answer is rare, $p = 0.02$, and the check appears strong, with $\mathrm{TPR} = 0.9$ and $\mathrm{FPR} = 0.05$. Then $\mathrm{prec} = 0.018/(0.018 + 0.049) = 0.27$: nearly three of every four "verified" answers are wrong. In terms of counts, out of 100 samples, 1.8 correct answers and 4.9 incorrect ones are accepted. With $\mathrm{FPR} = 0.005$, the precision rises to $0.018/(0.018 + 0.0049) = 0.79$. A false-positive rate roughly ten times smaller than the rate of the correct answer is what makes search worthwhile.

18.2 The verified vote and the shared-error condition

Returning any sample that passes the check discards the information contained in the vote. A better rule, the verified vote, groups the accepted samples by answer and returns the answer with the most accepted samples, falling back to the plain majority vote if no sample is accepted. Let π_a denote the fraction of samples with answer a, let the true answer t pass the check with probability TPR, and let each wrong answer w pass with its own rate FPR_w.

Key result | when a verified vote finds the truth

As $k \to \infty$, the verified vote returns the true answer if and only if

$$\pi_t\,\mathrm{TPR} \;>\; \max_{w \neq t}\; \pi_w\,\mathrm{FPR}_w,$$

and, for finite k, $\Pr[w \text{ ties or beats } t] \leq \exp\!\big(-\tfrac{k}{2}\,(\pi_t\mathrm{TPR} - \pi_w\mathrm{FPR}_w)^2\big)$.

Derivation. Answer a collects approximately $k\,\pi_a\,\Pr[\text{accept} \mid a]$ accepted samples, so by the law of large numbers the verified vote selects the answer with the largest $\pi_a \Pr[\text{accept} \mid a]$. For the finite-sample bound, we assign each sample a score of $+1$ if it answers t and is accepted, -1 if it answers w

and is accepted, and 0 otherwise. The mean score is $\pi_t \mathrm{TPR} - \pi_w \mathrm{FPR}_w$, each score lies in an interval of length 2, and Hoeffding's inequality yields the bound, exactly as for plurality voting in Section 17.

Worked example | the check must reject the wrong mode, not wrong answers in general

Suppose a shared error gives the wrong answer w half of the samples, $\pi_w = 0.5$, while the truth receives $\pi_t = 0.2$. With $\mathrm{TPR} = 0.9$, the verified vote recovers the truth only if $\mathrm{FPR}_w < 0.2 \times 0.9/0.5 = 0.36$; that is, the check must reject the *shared* wrong answer at least 64% of the time. A check that rereads the problem with the same misinterpretation accepts w almost always ($\mathrm{FPR}_w \approx 1$), and the verified vote then returns w, exactly as the plain majority vote does. The false-positive rate averaged over all wrong answers can look excellent even though FPR_w on the mode is the only quantity that matters.

Pitfall. A check's TPR and FPR should be reported *separately on the vote's wrong mode* (the questions on which the vote is wrong), not averaged over all questions. Signals tend to look better where the truth already holds the majority, so a probe or confidence score can appear strong overall yet fail within the minority-truth regime, which is the only place where it could help. Within that regime, it should be compared against the model's own token probability.

18.3 What additional samples buy: the coverage law

Brown et al. (*Large Language Monkeys*) distinguish two quantities. **Coverage** is "the fraction of problems that are solved by any generated sample" (pass@k, Section 7), whereas **precision** asks "how often can we identify correct samples from our collection of generations?"

Definition | the coverage law (Brown et al., Eqs. 2–3)

$$\log C \approx a\, k^b \qquad \Longleftrightarrow \qquad C \approx \exp(a\, k^b),$$

where C is the coverage, k the number of samples, and a, b are fitted parameters ($b < 0$, so $C \to 1$ as $k \to \infty$). Inverting the law gives the sample budget required for a target coverage: $k \approx (\ln C / a)^{1/b}$.

Worked example | Llama-3-8B-Instruct on MATH

The paper's fitted parameters are $a = -1.33$ and $b = -0.43$. The fit predicts a coverage of 0.26 at $k = 1$, 0.61 at $k = 10$, and 0.83 at $k = 100$ (the paper reports measured values of 82.9% at 100 samples and 98.44% at $10{,}000$). Reaching 90% coverage requires $k \approx (\ln 0.9 / -1.33)^{-1/0.43} \approx 364$ samples.

Key result | finding scales, selection saturates

On the same model and benchmark, coverage rises from 82.9% to 98.44% as the number of samples grows from 100 to $10{,}000$, whereas majority voting and reward-model selection improve only from 40.50% to 41.41%. All selection methods "saturate before reaching 100 samples", because "the occurrence of rare, correct solutions does not affect the most common answer that majority voting chooses"; such correct solutions are sampled "at low probabilities (e.g. 1% or lower)".

With an automatic verifier, coverage translates directly into accuracy: on SWE-bench Lite, DeepSeek-Coder-V2 improves from 15.9% with one sample to 56% with 250, exceeding the single-sample state of the art (43%). Moreover, rare correct answers are mostly genuine: over 90% of the graded chains of thought were faithful, even where correct answers were rare. In these settings the correct answer is present among the samples; what is missing is a reliable means of recognizing it.

18.4 Why an imperfect check caps search

Stroebl, Kapoor and Narayanan (*The Limits of Inference Scaling Through Resampling*) study code generation with unit tests that occasionally pass incorrect programs. They name the two error rates **completeness**, the "probability of accepting a correct solution" (our TPR), and **soundness**, the "probability of rejecting an incorrect solution" (our $1-\mathrm{FPR}$). As before, p denotes the probability that a single sample is correct (the paper denotes it r).

Key result | the resampling ceiling

Consider resampling until the check accepts. A false positive returned at any attempt is final, and "resampling cannot decrease this probability, so it imposes an upper bound to the accuracy of resampling-based inference scaling, regardless of compute budget". For a single task type, the accuracy of the returned answer equals the precision of Section 16,

$$\mathrm{prec} = \frac{p\,\mathrm{TPR}}{p\,\mathrm{TPR} + (1-p)\,\mathrm{FPR}},$$

regardless of the budget K. Consequently, if $P_{\text{strong}}(\text{correct}) > P_{\text{weak}}(\text{correct} \mid \text{pass})$, then "the weaker model cannot match the performance of a single invocation of the stronger model, no matter how big the compute budget".

Derivation. Each attempt terminates the search with a correct answer with probability $p\,\mathrm{TPR}$ and with an incorrect one with probability $(1-p)\,\mathrm{FPR}$; otherwise the search continues. Whenever the search terminates, the answer is correct with probability $p\,\mathrm{TPR}/(p\,\mathrm{TPR} + (1-p)\,\mathrm{FPR})$, so the accuracy is independent of K; additional attempts only increase the probability that the search terminates at all.

Worked example | easy and hard tasks

The paper's two-type model for Llama-3.1-8B on HumanEval+ has easy tasks with $p = 0.87$ and hard tasks with $p = 0.13$, with $\mathrm{TPR} = 1$ and $\mathrm{FPR} = 0.25$. On hard tasks, the precision is $0.13/(0.13 + 0.25 \times 0.87) = 0.37$, so most accepted answers are wrong. Each rejection raises the posterior probability that the task is hard: after $k-1$ rejections, $\Pr[\text{type } i] \propto \beta_i^{k-1} w_i$, where w_i is the fraction of tasks of type i and $\beta_i = (1-\mathrm{TPR})\,p_i + (1-\mathrm{FPR})(1-p_i)$ is its rejection rate. Consequently, the share of false positives *grows* with the number of attempts: "the easy tasks get solved within a few attempts, and for the remaining harder tasks, false positives are more likely". Empirically, the false-positive share of Llama-3.1-70B rises from about 0.13 at $K = 1$ to about 0.65 at large K, and when a false positive costs four times as much as a true positive gains, the optimal budget is $K \le 5$ for all four models tested.

Pitfall. The analysis of Stroebl et al. treats rejections as independent draws. A self-check whose errors track the model's own misreading behaves worse, because it accepts the *same* wrong answer repeatedly. This is the case $\mathrm{FPR}_w \approx 1$ of the verified-vote condition, from which no number of samples can escape.

18.5 Sources of strict checks

By the data processing inequality (Section 15), a check computed solely from the model's own outputs adds no new information. A check can help only by exploiting an *asymmetry*: either verification is easier than solving, or the check depends on something the error does not control (Section 17.8). We list four practical sources:

Source	Why it is easier than solving	Where it fails
Substitution	substituting 7 into $3x+5=26$ is trivial	applies only to answers that can be substituted
Backward check (FOBAR)	recovering a masked number from the answer reads the problem in the reverse direction	if the same misreading is reused, the check confirms it
Answer-first construction	the answer is fixed before the question is written	the written question may not match the intended problem
Execution (Absolute Zero)	a program's output is computed rather than judged	applies only to executable tasks (code)

18.6 The backward check in practice: FOBAR

FOBAR (Jiang et al.) verifies a candidate answer by inverting the question. Each number in the question is replaced in turn by x, and the model is asked:

"If we know the answer to the above question is $\{\hat{A}_j\}$, what is the value of unknown variable x?"

Definition | forward–backward scoring (FOBAR, Eqs. 1–4)

Let the candidate answers $\hat{A}_j$ be the distinct forward answers, forming the set $\mathcal{A}$. The forward score is the vote share, $\mathbb{P}_F(\hat{A}_j) = \frac{1}{M_F}\sum_{i=1}^{M_F} \mathbf{1}(A_i = \hat{A}_j)$. For the backward score, with N_Q masked numbers and M_B backward samples per number,

$$Z_j = \sum_{n=1}^{N_Q}\sum_{b=1}^{M_B} \mathbf{1}(\widehat{\text{num}}_{j,b}^{(n)} = \text{num}^{(n)}), \qquad \mathbb{P}_B(\hat{A}_j) = \frac{Z_j + \epsilon}{\sum_{j'} Z_{j'} + \epsilon|\mathcal{A}|},$$

and the final answer maximizes $\mathbb{P}(\hat{A}_j) \propto \mathbb{P}_F(\hat{A}_j)^{\alpha}\,\mathbb{P}_B(\hat{A}_j)^{1-\alpha}$, with $\alpha = 0.5$ ($\alpha = 1$ recovers plain self-consistency). We write j for the paper's candidate index c. The paper uses $M_B = 8$, temperature 0.7, and exact string matching.

Connection. With $\alpha = \frac{1}{2}$, the maximizer of $\sqrt{\mathbb{P}_F\,\mathbb{P}_B}$ coincides with the maximizer of $\mathbb{P}_F\,\mathbb{P}_B \propto \pi_a \Pr[\text{check passes} \mid a]$. FOBAR is therefore exactly the verified vote defined above, and its success condition is $\pi_t\,\text{TPR} > \pi_w\,\text{FPR}_w$.

Worked example | what the paper's numbers show

- **The check is weak.** A masked number is recovered 41–53% of the time from the correct candidate, but also 9–25% of the time from an incorrect one (GPT-4: 25.0% and 19.2%; Fig. 5). Hence $\text{TPR} \approx 0.5$ and $\text{FPR} \approx 0.1$–0.25 per backward sample.
- **Gains shrink as models become stronger.** Average accuracy over six datasets, self-consistency → FOBAR, is $81.4 \to 83.5$ (text-davinci-003), $85.5 \to 86.6$ (GPT-3.5), and $91.1 \to 91.4$ (GPT-4) (Table 1, CoT prompts).
- **The paper also exhibits the absent-answer ceiling.** For GPT-3.5, "about 60% of the total failure cases have at least one correct chain (the remaining 40% have no correct chains and thus cannot be solved by backward reasoning)"; FOBAR corrected 54 of those 294 cases (Sec. 4.8).
- **A shared misreading can pass the check.** In one example, the gold answer is 1.9 dollars, yet the backward score prefers 190 (cents) by 0.57 to 0.43 (App. C.2).

Substituting the paper's rates into the verified-vote condition, with $\text{TPR} = 0.5$ and $\text{FPR}_w = 0.2$, the truth wins only if $\pi_t > 0.4\,\pi_w$; against a wrong mode holding half of the samples, the truth needs at least 20% of them. The paper itself notes two limitations: masking a *redundant* number renders it unpredictable, and choosing what to mask is difficult outside mathematics.

18.7 Self-play with a perfect check: Absolute Zero

Absolute Zero (Zhao et al.) is the cleanest AlphaGo-like loop for language models: a single model both proposes and solves tasks, and a Python executor serves as the referee. Each task is a triplet (p, i, o) with $o = p(i)$, and three task types are defined:

Task	Given $\to$ predict	How the executor checks
Deduction	$(p, i) \to o$	value equality between the predicted and true outputs
Abduction	$(p, o) \to i$	$p(i_\pi) = o$; any input that produces the output is accepted, since programs need not be one-to-one
Induction	half of the (i, o) pairs and a message $\to p$	p_π must reproduce *held-out* pairs, which prevents memorization through if-else branches

Definition | rewards and advantages (Absolute Zero, Eqs. 4–8)

Proposer: $r_{\text{propose}} = 0$ if $\bar{r}_{\text{solve}} = 0$, and $1 - \bar{r}_{\text{solve}}$ otherwise, where $\bar{r}_{\text{solve}}$ is the mean over $G = 8$ solver attempts. Solver: $r_{\text{solve}} = \mathbf{1}(y = y^\star)$, evaluated by value equality in Python. Format penalties: -0.5 for a well-formatted wrong answer and -1 for a format error. Advantages are normalized with six separate baselines, one per (task, role) pair:

$$A^{\text{norm}}_{\text{task,role}} = \frac{r - \mu_{\text{task,role}}}{\sigma_{\text{task,role}}}, \qquad \text{task} \in \{\text{ind}, \text{ded}, \text{abd}\},\ \text{role} \in \{\text{propose}, \text{solve}\}.$$

A proposed task is retained only if the program runs, uses no banned packages, and produces the same output on two executions (a determinism check).

Worked example | what transfers, and what does not

AZR-Coder-7B raises the average over code and math benchmarks from 40.2 to 50.4, including a gain of $+15.2$ points in math, and outperforms earlier zero-data models by 1.8 points on average (Table 1). Its math score (39.1) nevertheless remains below that of zero-RL models trained on curated math data (for example, 45.8): the math gain is *transfer* from verified code, not from verified math. Gains increase with model size ($+5.7$, $+10.2$, $+13.2$ for 3B, 7B, 14B), and the authors report that composite programs collapsed to the trivial form $f(g(x)) = g(x)$, an instance of the proposer gaming the format of its own tasks. The lesson is that, with an executor, rare successes are recognized with $\text{FPR} \approx 0$. Natural-language math has no such executor, so the open question is how closely an inexpensive check can approach $\text{FPR}_w \approx 0$ on the shared wrong answer.

Why it matters for self-play. Taken together, these papers yield a single protocol for evaluating a new check. (1) **Find:** on the questions where the vote is wrong, we measure how often the truth appears among the samples and how this frequency grows with k (the coverage law). (2) **Recognize:** we measure the check's TPR on the truth and its FPR_w on the vote's wrong answer, rather than on average, and test $\pi_t \,\text{TPR} > \pi_w \,\text{FPR}_w$ question by question. (3) **Matched compute:** a check that costs as much as ten extra samples must outperform a vote with ten extra samples. (4) **The ceiling:** if the check accepts the shared wrong answer about as often as the truth appears, further search makes the wrong answer more certain, not less.

19 Model collapse

Key result | recursive training erodes the tails

Suppose each generation fits a Gaussian by maximum likelihood to N samples drawn from the previous generation's fit. Since $\mathbb{E}[\hat{\sigma}^2] = \frac{N-1}{N}\sigma^2$ for the maximum-likelihood variance estimate, iterating gives

$$\mathbb{E}[\sigma_t^2] = \Big(\frac{N-1}{N}\Big)^t \sigma_0^2 \;\longrightarrow\; 0.$$

Worked example | the rate of collapse

With $N = 100$ and $t = 100$, $(0.99)^{100} = 0.366$: two thirds of the variance has been lost, and the rare cases (the tails) are the first to disappear.

Shumailov et al. (2024) demonstrate the same loss of tails in language models trained recursively on their own outputs. Gerstgrasser et al. (2024) show that *accumulating* real and synthetic data, rather than replacing earlier data, keeps the error bounded.

Why it matters for self-play. R-Zero's Solver trains exclusively on labels it produced itself, which places it in the pure "replace" regime. The known remedies are retaining earlier data, anchoring to the base model, and incorporating external data.

20 Weak-to-strong generalization

Definition | performance gap recovered (Burns et al., 2023)

$$\mathrm{PGR} = \frac{\mathrm{Perf}_{\text{weak}\to\text{strong}} - \mathrm{Perf}_{\text{weak}}}{\mathrm{Perf}_{\text{strong ceiling}} - \mathrm{Perf}_{\text{weak}}}.$$

Strong models trained on a weaker model's labels often recover a substantial fraction of the gap to their ceiling performance: labels less accurate than the student can still teach it. This result is the optimistic counterpart to model collapse.

Why it matters for self-play. Noisy self-labels can still teach, provided that their errors are not ones the student already believes. Self-labels are risky precisely because their errors are, by construction, the student's own beliefs.

Exercises for Part V

Exercise 9. For the Condorcet model with $n = 3$ and $q = 0.55$, what is P_{maj}?

Exercise 10. A verifier has $\mathrm{TPR} = 0.8$ and $\mathrm{FPR} = 0.4$ on a problem set with $p = 0.2$. What is the accuracy of the answers it accepts?

Exercise 11. In the Gaussian collapse model with $N = 50$, what fraction of the variance remains after 50 generations?

Exercise 12. Samples have a pairwise error correlation of $r = 0.1$. How many independent votes are 50 samples worth, and what is the limiting value as the number of samples grows?

Exercise 13. The vote is wrong on 30% of questions. A new signal rescues 20% of the wrong votes and breaks 5% of the correct ones. Does overriding the vote help? Compute Δacc and ER.

Exercise 14. Half of the questions have $q = 0.9$ and the other half have $q = 0.3$. Compute the single-sample accuracy, the accuracy of an infinite vote, and the between-sample error correlation r.

Exercise 15. A question is solved by 3% of samples. How many samples are needed for a 90% chance of finding the correct answer at least once?

Exercise 16. A check has $\mathrm{TPR} = 0.8$ and $\mathrm{FPR} = 0.1$, and the correct answer appears in 5% of samples. What fraction of accepted answers is correct? What is the largest FPR that keeps this fraction at or above one half?

Exercise 17. The truth receives $\pi_t = 0.3$ of the samples and the vote's wrong answer receives $\pi_w = 0.4$. The check accepts the truth with $\mathrm{TPR} = 0.9$. Below what value of FPR_w on the wrong answer does the verified vote select the truth for large k?

PART VI

Evidence that reviewers trust

21 How large is benchmark noise?

Key result | benchmark uncertainty

For a benchmark of N problems with accuracy near p, the standard error is $\sqrt{p(1-p)/N}$, and a 95% confidence interval extends approximately ± 1.96 standard errors around the estimate.

Benchmark	Problems N	Standard error at $p = 0.5$	95% half-width
AIME (single year)	30	0.091	± 17.9 points
MATH-500	500	0.022	± 4.4 points
GSM8K (test split)	$1{,}319$	0.014	± 2.7 points

Averaging k samples per problem (avg@k) averages out sampling noise, but it does not address the variance that arises from which problems happen to be included in the benchmark. Two methods should therefore be compared with a *paired* bootstrap over the same set of problems.

22 A checklist before claiming results

- **Same compute:** match the number of rollouts, tokens and optimizer steps for every baseline.
- **Seeds:** use at least three seeds for training curves; a "rise, then fall" claim requires the decline to exceed the spread across seeds.
- **pass@k:** use the unbiased estimator with $n \gg k$, and report results up to large k.
- **Novelty:** report the base model's pass@256 and count the problems that the base model never solves.
- **Contamination:** check for exact and 13-gram overlap between generated problems and every evaluation benchmark.
- **Decision rules first:** specify the pass/fail criterion before the run, and never select checkpoints on test sets.
- **Read the code:** published numbers are produced by code, not by equations (Section 10).

PART VII

Open problems: directions for new work

1. **Sustained growth.** Pure self-play peaks after roughly three to five rounds, and later Challengers are no more instructive than earlier ones (*What Actually Evolves in Self-Evolving RL?*, ICLR 2027 submission). Collapse mitigations such as R-Diverse and data gating (Survive or Collapse) prevent the decline but not the plateau. *Goal:* a loop that continues to improve.
2. **Answer keys under shared errors.** The goal is a label-free signal that outperforms majority vote in the minority-truth zone (Section 17) by drawing on a channel that a shared misreading does not control (Sections 17.4 and 17.8).
3. **Search with a strict check.** The aim is to find rare correct answers and to recognize them without labels. The check's false-acceptance rate on the majority vote's incorrect answer must remain below the share of samples that carry the correct answer (Section 18).
4. **Measured expansion.** Progress should be demonstrated as newly solved problems at large k, beyond the base model's pass@256 (Section 7), rather than as pass@1 gains that may reflect only sharpening.
5. **Grounded teacher rewards without labels.** SOAR shows that rewarding actual student progress is effective, but it requires a labeled set of hard problems (Section 10).
6. **A theory of closed-loop improvement.** Under what conditions can a loop with no external information improve, and by how much? The data processing inequality (Section 15) and the verifier gap (Section 16) delineate the boundary.

A four-week study plan

Week	Study in this guide and derive by hand	Primary readings
1	Part I: policy gradient, baselines, the RLOO identity, PPO clipping	Sutton and Barto, ch. 13 [2]; PPO [3]; GRPO [4]; RLOO [5]
2	Part II: the tilted optimum, pass@k and its estimator	DPO derivation [8]; pass@k [9]; Yue et al. [10]; Dr. GRPO [7]
3	Part III: learnability, proposer rewards, noisy labels	Absolute Zero [15]; R-Zero [16]; SOAR [17]; PAIRED [12]
4	Parts IV–V: games, STaR, information limits, voting, search with a check, collapse	STaR [24]; Mind the Gap [28]; Large Language Monkeys [43]; resampling limits [44]; FOBAR [45]; model collapse [29, 30]; weak-to-strong [31]

A test of understanding. For each key result, the reader should be able to (1) derive it on a blank page, (2) reproduce its worked example without notes, and (3) state in one sentence its implication for a Challenger–Solver loop.

Appendix A Formula sheet

Idea	Formula	Section
Policy gradient	$\nabla_\theta J = \mathbb{E}[R\,\nabla_\theta \log \pi_\theta(y \mid x)]$	2
Baseline	$\mathbb{E}[b(x)\nabla_\theta \log \pi_\theta] = 0, \quad \hat{A} = R - b$	3
GRPO	$\hat{A}_i = (R_i - \text{mean}\, R)/(\text{std}\, R + \epsilon)$	4
RLOO	$\hat{A}_i = R_i - \frac{1}{G-1}\sum_{j\neq i} R_j = \frac{G}{G-1}(R_i - \text{mean}\, R)$	4
PPO clip	$\min\big(\rho\hat{A},\ \text{clip}(\rho, 1\pm\varepsilon)\hat{A}\big), \quad \rho = \pi_\theta/\pi_{\text{old}}$	5
KL estimator	$k_3 = r - 1 - \log r, \quad r = \pi_{\text{ref}}/\pi_\theta$	5
KL optimum	$\pi^\star \propto \pi_{\text{ref}}\, e^{R/\beta}; \quad \text{odds}^\star = \text{odds} \cdot e^{1/\beta}$	6
pass@k	$1-(1-p)^k; \quad 1 - \binom{n-c}{k}/\binom{n}{k}$	7
Learnability	$\text{Var}\, R = p(1-p); \quad$ informative group $1 - p^G - (1-p)^G$	9
R-Zero reward	$\max(0,\ 1 - 2\lvert\hat{p} - \frac{1}{2}\rvert - \lambda\lvert C_k\rvert/B)$	10
Absolute Zero	$1 - \bar{r}$ if $0 < \bar{r} < 1$, else 0	10
SOAR	$\text{Perf}_H(\text{Train}(\theta, D_\phi)) - \text{Perf}_H(\theta)$	10
PAIRED regret	$U(\pi_{\text{ant}}) - U(\pi_{\text{pro}})$	10
Symmetric label noise	$\mathbb{E}[\tilde{R}\,\nabla_\theta \log \pi] = (1-2\eta)\nabla_\theta J$	11
Bilinear spiral	$u^2 + v^2$ grows by $(1+\eta^2)$ per step	12
Fisher identity	$\nabla_\theta \log p(y^\star \mid x) = \mathbb{E}_{p(z\mid x, y^\star)}[\nabla_\theta \log p(z, y^\star \mid x)]$	14
RL vs. likelihood	$\nabla_\theta p = p\,\nabla_\theta \log p$	14
Data processing	$I(T;S) \le I(T;X)$ for $T \to X \to S$	15
Verifier gap	$\text{prec} = \frac{p\text{TPR}}{p\text{TPR} + (1-p)\text{FPR}}; \quad \text{prec} > p \iff \text{TPR} > \text{FPR}$	16
Condorcet	$\sum_{k>n/2}\binom{n}{k} q^k (1-q)^{n-k}; \quad \text{error} \le e^{-2n(q-1/2)^2}$	17.1
Plurality margin	$\Pr[w \text{ ties or beats } t] \le e^{-n(\pi_t - \pi_w)^2/2}$	17.2
Shared-error cap	$\rho q + (1-\rho)P_{\text{maj}}(n) \to \rho q + 1 - \rho$	17.3
Effective votes	$n_{\text{eff}} = n/(1 + (n-1)r) \to 1/r$	17.3
Voting ceiling	$\Pr_x[\text{mode of } \pi(\cdot \mid x) \text{ is the truth}]$	17.3
Indistinguishability	$O(x) \overset{d}{=} O(x'),\ t \neq t' \Rightarrow \Pr[L(x){=}t] + \Pr[L(x'){=}t'] \le 1$	17.4
Difficulty-induced correlation	$r = \text{Var}_x[q(x)]/\big(\bar{q}(1-\bar{q})\big)$	17.3
Override gain	$\Delta\text{acc} = ea - (1-e)b > 0 \iff a/b > (1-e)/e$	17.7
Samples needed to find	$1-(1-p)^k; \quad k_{90\%} = \ln 10/(-\ln(1-p)) \approx 2.3/p$	18
Strict check	$\text{prec} \ge \frac{1}{2} \iff \text{FPR} \le \frac{p}{1-p}\text{TPR}$	18.1
Verified vote	truth wins $\iff \pi_t\,\text{TPR} > \max_w \pi_w\,\text{FPR}_w$	18.2
Coverage law	$C \approx \exp(a\,k^b), \ \ b < 0$	18.3
Resampling ceiling	accuracy $\le$ prec (Section 16), for any budget K	18.4
FOBAR	$\mathbb{P} \propto \mathbb{P}_F^{\alpha}\,\mathbb{P}_B^{1-\alpha}, \ \ \alpha = \frac{1}{2}$	18.6
Model collapse	$\mathbb{E}[\sigma_t^2] = ((N-1)/N)^t \sigma_0^2$	19
Weak-to-strong	$\text{PGR} = \frac{\text{w2s}-\text{weak}}{\text{ceiling}-\text{weak}}$	20
Standard error	$\sqrt{p(1-p)/N}$	21

Appendix B Answers to the exercises

1. The mean is 0.75 and the standard deviation is 0.433, so the correct samples receive $+0.577$ and the incorrect one receives -1.732. On an easy problem, the rare failure is the most informative sample.
2. Without the division, $\hat{A}_i = R_i - \operatorname{mean} R = \frac{G-1}{G} \times \mathrm{RLOO}_i$. For $G = 8$, the factor is $7/8$.
3. The odds are $0.0101 \times e^5 = 0.0101 \times 148.4 = 1.50$, so $p^\star = 1.50/2.50 = 0.60$.
4. $1 - \binom{15}{8}/\binom{16}{8} = 1 - 6435/12870 = 0.5$. In general, $\binom{n-1}{k}/\binom{n}{k} = (n-k)/n$, so the estimate reduces to k/n.
5. $1 - 0.75^4 - 0.25^4 = 1 - 0.316 - 0.004 = 0.680$.
6. $1 - 2\eta = 0.4$. At $\eta = 0.5$ the factor is 0, so the labels carry no signal.
7. The squared radius grows by a factor of 1.01 per step, so the radius grows by $\sqrt{1.01^{10}} = 1.051$.
8. $1/p = 20$.
9. $q^3 + 3q^2(1-q) = 0.166 + 0.408 = 0.575$.
10. $0.16/(0.16 + 0.32) = 0.333$, which exceeds the base rate of 0.2; filtering therefore helps.
11. $(49/50)^{50} = e^{50 \ln 0.98} = 0.364$.
12. $n_{\text{eff}} = 50/(1 + 49 \times 0.1) = 8.5$, and the limit is $1/r = 10$.
13. The required ratio is $0.7/0.3 = 2.33$; here $a/b = 4$, so the condition holds. $\Delta\text{acc} = 0.3 \times 0.2 - 0.7 \times 0.05 = 0.025$ (2.5 points), and $\mathrm{ER} = 0.2 - 0.05 \times 2.33 = 0.083$.
14. A single sample achieves $\bar{q} = 0.6$, whereas an infinite vote achieves 0.5 (only the easy half). Since $\mathrm{Var}[q] = 0.45 - 0.36 = 0.09$, we obtain $r = 0.09/0.24 = 0.375$.
15. $k = \ln 10/(-\ln 0.97) = 2.303/0.0305 = 75.6$, so 76 samples are required (the rule of thumb $2.3/p$ gives 77).
16. $\mathrm{prec} = 0.04/(0.04 + 0.095) = 0.30$. Precision reaches one half when $\mathrm{FPR} \le \frac{p}{1-p}\mathrm{TPR} = \frac{0.05}{0.95} \times 0.8 = 0.042$.
17. $\mathrm{FPR}_w < \pi_t \,\mathrm{TPR}/\pi_w = 0.27/0.4 = 0.675$; that is, the check must reject the incorrect answer in at least roughly one third of cases.

References

[1] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 1992.
[2] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
[3] J. Schulman et al. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017.
[4] Z. Shao et al. DeepSeekMath (introduces GRPO). arXiv:2402.03300, 2024.
[5] A. Ahmadian et al. Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs (RLOO). arXiv:2402.14740, 2024.
[6] Q. Yu et al. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. arXiv:2503.14476, 2025.
[7] Z. Liu et al. Understanding R1-Zero-Like Training: A Critical Perspective (Dr. GRPO). arXiv:2503.20783, 2025.
[8] R. Rafailov et al. Direct Preference Optimization. arXiv:2305.18290, 2023.
[9] M. Chen et al. Evaluating Large Language Models Trained on Code (unbiased pass@k). arXiv:2107.03374, 2021.
[10] Y. Yue et al. Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model? arXiv:2504.13837, 2025.
[11] S. Sukhbaatar et al. Intrinsic Motivation and Automatic Curricula via Asymmetric Self-Play. arXiv:1703.05407, 2017.
[12] M. Dennis et al. Emergent Complexity and Zero-shot Transfer via Unsupervised Environment Design (PAIRED). arXiv:2012.02096, 2020.
[13] J. Schmidhuber. PowerPlay: Training an Increasingly General Problem Solver by Continually Searching for the Simplest Still Unsolvable Problem. arXiv:1112.5309, 2011.
[14] P.-Y. Oudeyer, F. Kaplan and V. Hafner. Intrinsic motivation systems for autonomous mental development. *IEEE Trans. Evolutionary Computation*, 2007.
[15] A. Zhao et al. Absolute Zero: Reinforced Self-play Reasoning with Zero Data. arXiv:2505.03335, 2025.

[16] C. Huang et al. R-Zero: Self-Evolving Reasoning LLM from Zero Data. arXiv:2508.05004, 2025.
[17] S. Sundaram, J. Kempe and Y. Ollivier. Teaching Models to Teach Themselves: Reasoning at the Edge of Learnability (SOAR). arXiv:2601.18778, ICML 2026.
[18] R-Diverse: Mitigating Diversity Illusion in Self-Play LLM Training. arXiv:2602.13103, 2026.
[19] Survive or Collapse: The Asymmetric Roles of Data Gating and Reward Grounding in Self-Play RL. arXiv:2605.22217, 2026.
[20] J. Robinson. An iterative method of solving a game. *Annals of Mathematics*, 1951.
[21] M. Lanctot et al. A Unified Game-Theoretic Approach to Multiagent Reinforcement Learning (PSRO). arXiv:1711.00832, 2017.
[22] T. Anthony, Z. Tian and D. Barber. Thinking Fast and Slow with Deep Learning and Tree Search (expert iteration). arXiv:1705.08439, 2017.
[23] D. Silver et al. Mastering the game of Go without human knowledge. *Nature*, 2017; and Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm, arXiv:1712.01815.
[24] E. Zelikman et al. STaR: Bootstrapping Reasoning With Reasoning. arXiv:2203.14465, 2022.
[25] X. Wang et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv:2203.11171, 2022.
[26] Y. Zuo et al. TTRL: Test-Time Reinforcement Learning. arXiv:2504.16084, 2025.
[27] T. M. Cover and J. A. Thomas. *Elements of Information Theory*, 2nd ed. Wiley, 2006.
[28] Y. Song et al. Mind the Gap: Examining the Self-Improvement Capabilities of Large Language Models. arXiv:2412.02674, ICLR 2025.
[29] I. Shumailov et al. AI models collapse when trained on recursively generated data. *Nature*, 2024 (arXiv:2305.17493).
[30] M. Gerstgrasser et al. Is Model Collapse Inevitable? Breaking the Curse of Recursion by Accumulating Real and Synthetic Data. arXiv:2404.01413, 2024.
[31] C. Burns et al. Weak-to-Strong Generalization: Eliciting Strong Capabilities With Weak Supervision. arXiv:2312.09390, 2023.
[32] D. Silver and R. S. Sutton. Welcome to the Era of Experience. 2025.
[33] W. Hoeffding. Probability inequalities for sums of bounded random variables. *J. American Statistical Association*, 1963.
[34] L. Kish. *Survey Sampling*. Wiley, 1965 (design effect).
[35] A. P. Dawid and A. M. Skene. Maximum likelihood estimation of observer error-rates using the EM algorithm. *Applied Statistics*, 1979.
[36] D. Prelec. A Bayesian truth serum for subjective data. *Science*, 2004.
[37] D. Prelec, H. S. Seung and J. McCoy. A solution to the single-question crowd wisdom problem. *Nature*, 2017.
[38] E. Kim, A. Garg, K. Peng and N. Garg. Correlated Errors in Large Language Models. arXiv:2506.07962, ICML 2025.
[39] C. Burns, H. Ye, D. Klein and J. Steinhardt. Discovering Latent Knowledge in Language Models Without Supervision. arXiv:2212.03827, 2022.
[40] H. Orgad et al. LLMs Know More Than They Show: On the Intrinsic Representation of LLM Hallucinations. arXiv:2410.02707, 2024.
[41] Beyond Majority Voting: LLM Aggregation by Leveraging Higher-Order Information. arXiv:2510.01499, 2025.
[42] Mitigating LLM sycophancy with RL-based fine-tuning: Bayesian Truth Serum approach. arXiv:2608.25267, 2026.
[43] B. Brown et al. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. arXiv:2407.21787, 2024.
[44] B. Stroebl, S. Kapoor and A. Narayanan. The Limits of Inference Scaling Through Resampling. arXiv:2411.17501, 2024.
[45] W. Jiang et al. Forward-Backward Reasoning in Large Language Models for Mathematical Verification (FOBAR). arXiv:2308.07758, 2023.